

So it appears we have a new primitive, `flatMap`, which enables context-sensitive parsing and lets us implement `map` and `map2`. This is not the first time `flatMap` has made an appearance.

We now have an even smaller set of just six primitives: `string`, `regex`, `slice`, `succeed`, `or`, and `flatMap`. But we also have more power than before. With `flatMap`, instead of the less-general `map` and `product`, we can parse not just arbitrary context-free grammars like JSON, but context-sensitive grammars as well, including extremely complicated ones like C++ and PERL!

9.4 Writing a JSON parser

Let's write that JSON parser now, shall we? We don't have an implementation of our algebra yet, and we've yet to add any combinators for good error reporting, but we can deal with these things later. Our JSON parser doesn't need to know the internal details of how parsers are represented. We can simply write a function that produces a JSON parser using only the set of primitives we've defined and any derived combinators.

That is, for some JSON parse result type (we'll explain the JSON format and the parse result type shortly), we'll write a function like this:

```
def jsonParser[Err, Parser[+_]] (P: Parsers[Err, Parser]): Parser[JSON] = {
  import P._
  val spaces = char(' ').many.slice
  ...
}
```



Gives access to all the combinators

This might seem like a peculiar thing to do, since we won't actually be able to run our parser until we have a concrete implementation of the `Parsers` interface. But we'll proceed, because in FP, it's common to define an algebra and explore its expressiveness without having a concrete implementation. A concrete implementation can tie us down and makes changes to the API more difficult. Especially during the design phase of a library, it can be much easier to refine an algebra *without* having to commit to any particular implementation, and part of our goal here is to get you comfortable with this style of working.

After this section, we'll return to the question of adding better error reporting to our parsing API. We can do this without disturbing the overall structure of the API or changing our JSON parser very much. And we'll also come up with a concrete, runnable representation of our `Parser` type. Importantly, the JSON parser we'll implement in this next section will be completely independent of that representation.

9.4.1 The JSON format

If you aren't already familiar with the JSON format, you may want to read Wikipedia's description (<http://mng.bz/DpNA>) and the grammar specification (<http://json.org>). Here's an example JSON document:

```
{
  "Company name" : "Microsoft Corporation",
  "Ticker"      : "MSFT",
```